

Practice Assignment: Naive Bayes Theorem

Course: PGD in Data Science with AI

Topic: Naive Bayes Classification

Attached Files Used: naive_bayes_tennis_simple.ipynb and naive_bayes_banking_loan_dataset.csv

Purpose: This assignment helps students understand Bayes' Theorem, conditional probability, and simple Naive Bayes implementation using the Play Tennis example and a banking loan approval dataset.

Learning Outcomes

- Understand Bayes' Theorem and the idea of prior, likelihood, posterior probability.
- Manually calculate simple Naive Bayes probabilities using the Play Tennis example.
- Apply Naive Bayes using simple Python code.
- Use a real banking dataset to train and test a Naive Bayes classifier.
- Interpret accuracy, confusion matrix, and classification report.

Part A: Conceptual Questions

1. Define Bayes' Theorem in your own words.
2. What is the meaning of prior probability $P(y)$?
3. What is likelihood $P(X|y)$?
4. What is posterior probability $P(y|X)$?
5. Why is it called 'Naive' Bayes?
6. Why is Naive Bayes mostly used for classification problems?
7. Give one health science example and one banking example where Naive Bayes can be used.

Part B: Manual Naive Bayes - Play Tennis Example

Use the Play Tennis dataset from the attached notebook. The target column is PlayTennis with two classes: Yes and No.

Dataset features include Outlook, Temperature, Humidity, and Wind.

Task: Manually predict whether tennis will be played for the following condition:

Outlook = Sunny, Temperature = Hot, Humidity = High, Wind = Weak

Steps to complete:

1. Calculate $P(\text{Yes})$ and $P(\text{No})$.
2. Calculate conditional probabilities such as $P(\text{Sunny}|\text{Yes})$, $P(\text{Hot}|\text{Yes})$, $P(\text{High}|\text{Yes})$, and $P(\text{Weak}|\text{Yes})$.
3. Calculate the score for Yes.
4. Calculate the score for No.
5. Compare both scores and write the final prediction.

Simple Manual Code for Tennis Example

```
import pandas as pd
```

[illegible]

Part C: Banking Loan Approval Dataset

Use the attached CSV file: `naive_bayes_banking_loan_dataset.csv`

The dataset contains the following columns:

income level, credit score, employment status, existing debt, previous default, loan status

Target column: loan status

Input features: all columns except loan_status

First 5 rows of attached banking dataset:

income_level	credit_score	employment_status	existing_debt	previous_default	loan_status
Low	Poor	Employed	Medium	No	Approved
Medium	Average	Employed	Low	No	Approved
Low	Good	Employed	High	Yes	Rejected
Medium	Average	Self-Employed	High	No	Approved
Medium	Poor	Unemployed	Medium	No	Rejected

Tasks:

1. Load the dataset using pandas.
2. Display the first 5 rows.
3. Check the number of rows and columns.
4. Check the class distribution of `loan_status`.
5. Split data into X and y.
6. Train a Naive Bayes model.

7. Predict test data.
8. Evaluate using accuracy, confusion matrix, and classification report.
9. Predict a new loan applicant.

Simple Code for Banking Dataset

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import CategoricalNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

df = pd.read_csv("naive_bayes_banking_loan_dataset.csv")

print(df.head())
print(df.shape)
print(df['loan_status'].value_counts())

X = df.drop('loan_status', axis=1)
y = df['loan_status']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = Pipeline([
    ('encoder', OrdinalEncoder()),
    ('classifier', CategoricalNB())
])

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

Predict a New Loan Applicant

```
new_applicant = pd.DataFrame({
    'income_level': ['Medium'],
    'credit_score': ['Good'],
    'employment_status': ['Employed'],
    'existing_debt': ['Low'],
    'previous_default': ['No']
})

prediction = model.predict(new_applicant)
probability = model.predict_proba(new_applicant)

print("Prediction:", prediction[0])
print("Class Labels:", model.named_steps['classifier'].classes_)
print("Probabilities:", probability)
```

Part D: Interpretation Questions

1. What does the accuracy score tell you?
2. Why is the confusion matrix useful?
3. In banking, which error is more serious: approving a risky applicant or rejecting a good applicant? Explain.

4. Why is Naive Bayes suitable for categorical data?
5. What limitation does Naive Bayes have because of its independence assumption?

Submission Requirements

- Submit your Python code or notebook.
- Submit screenshots of output including accuracy, confusion matrix, and classification report.
- Submit your manual Play Tennis calculation.
- Write a short conclusion comparing the manual tennis example and the banking model.

Short Conclusion Guide

Your conclusion should mention that Naive Bayes uses probability to classify data. It is simple, fast, and useful for categorical datasets such as weather-based tennis prediction and banking loan approval. However, its main limitation is that it assumes all features are independent, which may not always be true in real-world datasets.